
Linear Algebra for Machine Learning

Refresher Module 2 — Vectors, Transforms, Eigen & SVD

dot products & projection · Gram–Schmidt · the transformation zoo · orthonormal frames · change of basis · eigen/SVD · PCA · PageRank

Computing, Robotics & Bionics

3rd-year Bioengineering — companion to A. Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*

*This is the second of four refresher modules. It develops the geometric linear algebra that machine learning runs on: vectors, the dot product and projection, Gram–Schmidt orthonormalization, matrices read as maps that move space (scaling, rotation, reflection, shear, projection), orthonormal matrices and rigid motions, the determinant, change of basis, and the two factorisations that power ML — the eigendecomposition and the singular value decomposition — ending in least squares, principal-component analysis, and PageRank as an eigenvector-centrality case study. It is **self-contained**: together with its three companion Colab notebooks — a bioengineering-adapted superset of Géron’s `math_linear_algebra` material — it is your complete reference, so you never read two versions side by side. The same rotations and orthonormal frames appear in the companion Robotics & Bionics unit ($SO(3)$, homogeneous transforms); projection underlies least squares (Géron Ch. 4) and the eigen/SVD machinery underlies PCA (Ch. 7). Each section ends with exercises.*

Prerequisites: Module 1 (NumPy); high-school algebra and trigonometry. Software: a web browser and a Google account (everything runs in Google Colab; nothing to install).

Contents

1	Vectors and the Dot Product	2
1.1	The scalar product in a general basis	2
1.2	Orthonormal bases and the dot product formula	2
1.3	Length: the L2 norm	2
1.4	Geometric meaning: angle, orthogonality, cosine similarity	3
1.5	Projection	3
1.6	Gram–Schmidt orthonormalization	4
2	Matrices as Linear Maps	5
2.1	Columns are images of the basis vectors	5
2.2	Composition is matrix multiplication	5
3	The Transformation Zoo	5
4	Orthogonal Matrices and Rigid Motions	6
5	Determinant, Inverse, Rank	7
6	Change of Basis	7
7	Eigenvalues and Eigenvectors	8
8	The Singular Value Decomposition	8
9	Two Payoffs: Least Squares and PCA	8
9.1	Linear regression is a projection	8
9.2	PCA is an eigen/SVD problem	9
10	PageRank: An Eigenvector-Centrality Case Study	9
10.1	Importance as a fixed point	9
10.2	Finding $\mathbf{r}$: power iteration	9
10.3	Dead ends, traps and damping	10
11	The Lab, and Where This Leads in Géron	10

1 Vectors and the Dot Product

Machine learning represents a sample as a **vector** of features and a model as a set of **matrices**; training and prediction are then linear-algebra operations. This module builds the geometric picture behind them. We work in $\mathbb{R}^n$, writing a vector as a column $\mathbf{x} = (x_1, \dots, x_n)^\top$.

1.1 The scalar product in a general basis

Coordinates are never absolute — $(v_1, \dots, v_n)$ is a list of numbers *relative to a chosen basis* $\mathbf{e}_1, \dots, \mathbf{e}_n$, via $\mathbf{v} = \sum_i v_i \mathbf{e}_i$. The **scalar (inner) product** $\langle \mathbf{v}, \mathbf{w} \rangle$ is a number measuring how two vectors relate — their alignment and, as we will see, their lengths — and it must be computable from coordinates alone. Expanding both vectors bilinearly in the basis,

$$\langle \mathbf{v}, \mathbf{w} \rangle = \left\langle \sum_i v_i \mathbf{e}_i, \sum_j w_j \mathbf{e}_j \right\rangle = \sum_{i=1}^n \sum_{j=1}^n v_i w_j \langle \mathbf{e}_i, \mathbf{e}_j \rangle = \mathbf{v}^\top \mathbf{G} \mathbf{w}, \quad (1.1)$$

Definition

The matrix $\mathbf{G}$ is the basis's **Gram matrix**, $G_{ij} = \langle \mathbf{e}_i, \mathbf{e}_j \rangle$ — it records how every basis vector relates to every other one, and it is exactly the extra information the raw coordinates v_i, w_j do not carry.

If the basis is *not* orthonormal, $\mathbf{G}$ is some matrix other than the identity, and the scalar product is *not* $\sum_i v_i w_i$ — forgetting this is one of the most common sources of confusion in linear algebra. Equation (1.1) is the general rule; every formula in the rest of this module is the special case that follows next.

1.2 Orthonormal bases and the dot product formula

A basis is **orthonormal** when its vectors are mutually perpendicular and each has unit length, $\langle \mathbf{e}_i, \mathbf{e}_j \rangle = \delta_{ij}$ (1 if $i = j$, 0 otherwise) — i.e. exactly when $\mathbf{G} = \mathbf{I}$. Equation (1.1) then collapses to

$$\langle \mathbf{v}, \mathbf{w} \rangle = \mathbf{v}^\top \mathbf{w} = \sum_{i=1}^n v_i w_i, \quad (1.2)$$

the familiar **dot product** — a consequence of orthonormality, not a definition in its own right. $\mathbb{R}^n$'s standard basis $(1, 0, \dots, 0), \dots, (0, \dots, 0, 1)$ is orthonormal, so whenever we write coordinates in it (as we do throughout this module unless said otherwise), (1.2) is exactly the right formula, and we will call $\mathbf{v} \cdot \mathbf{w} = \mathbf{v}^\top \mathbf{w}$ “the dot product” from here on.

Row times column is bookkeeping, not the point

Writing $\mathbf{v}^\top \mathbf{w}$ as ordinary matrix multiplication needs $\mathbf{v}^\top$ to be a *row* and $\mathbf{w}$ a *column* — but that is a convention for multiplying matrices, not a property of the scalar product itself, which does not care how you arrange the numbers. In NumPy this is visible directly: a 1-D array has no row or column orientation at all, `x.T` does nothing to it, and `x @ y` computes (1.2) regardless. The row/column bookkeeping only becomes necessary once you commit to representing vectors as literal $n \times 1$ or $1 \times n$ matrices.

1.3 Length: the L2 norm

A vector's **length** is its scalar product with itself, square-rooted:

$$\|\mathbf{v}\| = \sqrt{\langle \mathbf{v}, \mathbf{v} \rangle}. \quad (1.3)$$

In the orthonormal/standard-basis case (1.2) makes this the **Euclidean (L2) norm**,

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + \cdots + x_n^2} = \sqrt{\mathbf{x}^\top \mathbf{x}}, \quad (1.4)$$

and the distance between two samples is $\|\mathbf{x} - \mathbf{y}\|_2$. Two other norms appear in ML: the **L1 norm** $\|\mathbf{x}\|_1 = \sum_i |x_i|$ (used by Lasso regularisation) and the **L ∞ norm** $\|\mathbf{x}\|_\infty = \max_i |x_i|$. A vector of length one is a **unit vector**, $\hat{\mathbf{x}} = \mathbf{x}/\|\mathbf{x}\|_2$.

1.4 Geometric meaning: angle, orthogonality, cosine similarity

The dot product's geometric meaning is the key to everything that follows:

$$\mathbf{x}^\top \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta, \quad (1.5)$$

where θ is the angle between the vectors. Three consequences matter:

- **Orthogonality.** $\mathbf{x}^\top \mathbf{y} = 0$ exactly when the vectors are perpendicular ($\cos 90^\circ = 0$). Orthogonality is just “dot product zero”.
- **Angle / similarity.** Rearranging (1.5) gives the **cosine similarity** $\cos \theta = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$, the standard measure of how aligned two feature vectors (or two gene-expression profiles, or two documents) are.
- **Sign.** The dot product is positive when vectors point the same way, negative when opposed — the basis of every linear classifier's decision $\mathbf{w}^\top \mathbf{x} + b \gtrless 0$.

Derived, or defined?

In $\mathbb{R}^2$ or $\mathbb{R}^3$, (1.5) is a *theorem*, not a definition: with θ the ordinary, protractor-measured angle, the Law of Cosines gives $\|\mathbf{x} - \mathbf{y}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{y}\|^2 - 2\|\mathbf{x}\|\|\mathbf{y}\|\cos \theta$, and expanding $\|\mathbf{x} - \mathbf{y}\|^2 = \mathbf{x}^\top \mathbf{x} - 2\mathbf{x}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}$ algebraically and comparing the two gives (1.5) straight back out. The genuinely surprising content is that the coordinate sum $\sum_i x_i y_i$ — which never mentions an angle — equals the classical geometric quantity $\|\mathbf{x}\|\|\mathbf{y}\|\cos \theta$, and this only works because the coordinates are orthonormal (§1.2). Once there is no protractor to appeal to — $\mathbb{R}^n$ for $n \geq 4$, or any abstract inner-product space — there is no independent angle left to derive a formula *for*. There, (1.5) is turned around and used as the **definition** of θ ,

$$\cos \theta := \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \|\mathbf{y}\|},$$

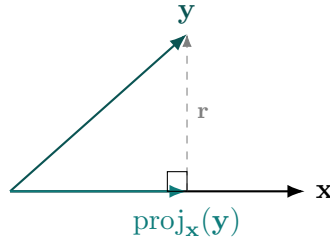
which only makes sense because the **Cauchy–Schwarz inequality** $|\langle \mathbf{x}, \mathbf{y} \rangle| \leq \|\mathbf{x}\|\|\mathbf{y}\|$ guarantees that ratio always lies in $[-1, 1]$, so arccos of it is always well defined.

1.5 Projection

Decomposing one vector along another is the geometric heart of least squares. The **orthogonal projection** of $\mathbf{y}$ onto the line through $\mathbf{x}$ is

$$\text{proj}_{\mathbf{x}}(\mathbf{y}) = \frac{\mathbf{x}^\top \mathbf{y}}{\mathbf{x}^\top \mathbf{x}} \mathbf{x} = (\hat{\mathbf{x}}^\top \mathbf{y}) \hat{\mathbf{x}}, \quad (1.6)$$

and the leftover $\mathbf{r} = \mathbf{y} - \text{proj}_{\mathbf{x}}(\mathbf{y})$ is **orthogonal** to $\mathbf{x}$. So any vector splits uniquely into a part along $\mathbf{x}$ and a part perpendicular to it.



Key idea

Projection onto a *subspace* (a plane, or the column space of a matrix) works the same way and is exactly how linear regression finds the best fit: it projects the target vector onto the space spanned by the feature columns, making the residual orthogonal to every feature. We derive that “normal equation” in Section 9.

1.6 Gram–Schmidt orthonormalization

An orthonormal basis — mutually perpendicular unit vectors — is the most convenient basis there is: as §1.2 showed, coordinates are dot products *exactly because* $\mathbf{G} = \mathbf{I}$ for such a basis, and (Section 6) the change-of-basis matrix built from it is orthogonal, so its inverse is free. Most bases you are handed are not orthonormal, but **Gram–Schmidt** turns any linearly independent set $\mathbf{v}_1, \dots, \mathbf{v}_k$ into an orthonormal set $\mathbf{q}_1, \dots, \mathbf{q}_k$ spanning the same subspace, using nothing but projection, one vector at a time:

$$\mathbf{q}_1 = \frac{\mathbf{v}_1}{\|\mathbf{v}_1\|}, \quad \mathbf{u}_k = \mathbf{v}_k - \sum_{j=1}^{k-1} (\mathbf{q}_j^\top \mathbf{v}_k) \mathbf{q}_j, \quad \mathbf{q}_k = \frac{\mathbf{u}_k}{\|\mathbf{u}_k\|}. \quad (1.7)$$

Each $\mathbf{u}_k$ is $\mathbf{v}_k$ with its projection (1.6) onto every previously built axis $\mathbf{q}_1, \dots, \mathbf{q}_{k-1}$ subtracted off — exactly the “along/perpendicular” split of Section 1.5, applied repeatedly against a growing subspace instead of a single line. What is left, $\mathbf{u}_k$, is perpendicular to everything built so far; normalising it gives the next axis.

Key idea

Stacking the $\mathbf{q}_k$ as columns of a matrix $\mathbf{Q}$ and recording the projection coefficients from (1.7) in an upper-triangular $\mathbf{R}$ gives the **QR decomposition** $\mathbf{A} = \mathbf{QR}$ of the matrix whose columns were $\mathbf{v}_1, \dots, \mathbf{v}_k$. Solving the least-squares normal equation of Section 9 via QR ($\mathbf{R}\mathbf{w} = \mathbf{Q}^\top \mathbf{y}$) avoids ever forming $\mathbf{X}^\top \mathbf{X}$, and is the numerically stable way real solvers do it.

Exercises

1. Let $\mathbf{e}_1 = (1, 0)^\top$, $\mathbf{e}_2 = (1, 1)^\top$ be a (non-orthonormal) basis of $\mathbb{R}^2$. Write out its Gram matrix $\mathbf{G}$, then compute $\langle \mathbf{v}, \mathbf{w} \rangle$ for $\mathbf{v} = 2\mathbf{e}_1 + \mathbf{e}_2$, $\mathbf{w} = \mathbf{e}_1 - \mathbf{e}_2$ two ways: via $\mathbf{v}^\top \mathbf{G} \mathbf{w}$ in $\mathbf{e}_1, \mathbf{e}_2$ coordinates, and via the ordinary dot product after writing $\mathbf{v}, \mathbf{w}$ out in standard coordinates. Confirm the two answers agree.
2. For $\mathbf{x} = (3, 4)^\top$, compute $\|\mathbf{x}\|_2$, $\|\mathbf{x}\|_1$, $\|\mathbf{x}\|_\infty$ and the unit vector $\hat{\mathbf{x}}$.
3. Are $(1, 2, 2)^\top$ and $(2, 1, -2)^\top$ orthogonal? Compute their cosine similarity.
4. Project $\mathbf{y} = (2, 3)^\top$ onto $\mathbf{x} = (1, 0)^\top$ and onto $\mathbf{x} = (1, 1)^\top$; give the residual in each case and verify it is orthogonal.
5. Explain in one sentence why a linear classifier’s rule $\mathbf{w}^\top \mathbf{x} + b > 0$ is a statement about

the angle between $\mathbf{w}$ and $\mathbf{x}$.

6. Apply Gram–Schmidt (1.7) to $\mathbf{v}_1 = (1, 1, 0)^\top$, $\mathbf{v}_2 = (1, 0, 1)^\top$ to find an orthonormal basis $\mathbf{q}_1, \mathbf{q}_2$ for their span, and confirm $\mathbf{q}_1^\top \mathbf{q}_2 = 0$.
7. Explain why solving the Section 9 normal equation via a QR factorisation is preferred in practice to inverting $\mathbf{X}^\top \mathbf{X}$ directly.

2 Matrices as Linear Maps

A matrix is most usefully read not as a grid of numbers but as a **function that transforms space**. Multiplying a vector by a matrix $\mathbf{A}$ sends $\mathbf{x}$ to a new vector $\mathbf{Ax}$, and this map is **linear**: it preserves lines through the origin and vector addition, $\mathbf{A}(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha\mathbf{Ax} + \beta\mathbf{Ay}$.

2.1 Columns are images of the basis vectors

The single most useful fact: *the columns of $\mathbf{A}$ are where it sends the standard basis vectors*. In 2-D, $\mathbf{A}\mathbf{e}_1$ is the first column and $\mathbf{A}\mathbf{e}_2$ the second, because

$$\mathbf{Ax} = \begin{bmatrix} | & | \\ \mathbf{a}_1 & \mathbf{a}_2 \\ | & | \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2. \quad (2.1)$$

So a matrix-vector product is a **weighted sum of the columns** (a linear combination), and to know what a matrix does to all of space it is enough to know what it does to the basis. This is how you can read a transformation straight off its columns, and how you build one to do what you want.

2.2 Composition is matrix multiplication

Applying $\mathbf{B}$ then $\mathbf{A}$ to a vector is $\mathbf{A}(\mathbf{Bx}) = (\mathbf{AB})\mathbf{x}$: **matrix multiplication is composition of maps**, performed right-to-left. It does not commute ($\mathbf{AB} \neq \mathbf{BA}$ in general) for the same reason physical motions do not: rotating then reflecting differs from reflecting then rotating. A neural-network layer $\mathbf{h} = \sigma(\mathbf{Wx} + \mathbf{b})$ is one such linear map (the $\mathbf{Wx}$ part) followed by a shift and a nonlinearity.

The mental switch

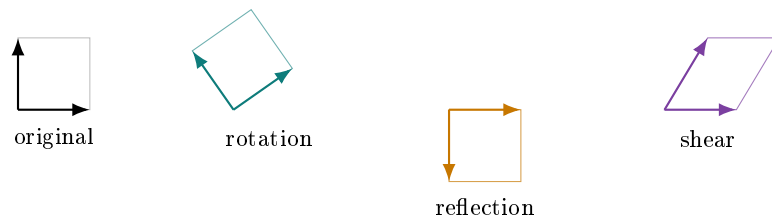
For the rest of this module, read every matrix as a verb. “What does this matrix *do* to the plane?” is answered by looking at where it sends $\mathbf{e}_1$ and $\mathbf{e}_2$ — its two columns. The next section catalogues the verbs you will meet.

3 The Transformation Zoo

Here are the linear maps you will meet, in 2-D, each read through its columns (where it sends $\mathbf{e}_1$ and $\mathbf{e}_2$).

Map	Matrix	What it does
Scaling	$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$	stretches axis x by s_x , axis y by s_y
Rotation by θ	$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$	turns every vector by θ about the origin
Reflection (in x -axis)	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	flips across the x -axis (a mirror)
Shear (horizontal)	$\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$	slides points sideways in proportion to height
Projection (onto x -axis)	$\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$	flattens the plane onto a line (information lost)

The rotation matrix is worth memorising; it is the 2-D case of the rotation matrices $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$ in the Robotics & Bionics unit. Note the projection's second column is zero: it sends $\mathbf{e}_2$ to the origin, so a whole direction collapses — projections are *not* invertible.



Exercises

- Write the matrix that rotates by 90° . Apply it to $(1,0)^\top$ and $(0,1)^\top$ and confirm the result matches its columns.
- What single matrix scales x by 2, leaves y unchanged, *and* then rotates by 90° ? (Remember: compose right-to-left.)
- Show that the projection $\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$ applied twice equals itself (it is *idempotent*), and explain why geometrically.

4 Orthogonal Matrices and Rigid Motions

A matrix $\mathbf{Q}$ is **orthogonal** if its columns are mutually perpendicular unit vectors, equivalently

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{I} \iff \mathbf{Q}^{-1} = \mathbf{Q}^\top. \quad (4.1)$$

Such maps **preserve lengths and angles**: $\|\mathbf{Q}\mathbf{x}\| = \|\mathbf{x}\|$ and $(\mathbf{Q}\mathbf{x})^\top (\mathbf{Q}\mathbf{y}) = \mathbf{x}^\top \mathbf{y}$. They are the *rigid motions* — they move a shape without distorting it. Their determinant is

$$\det \mathbf{Q} = \pm 1: \quad +1 \text{ a rotation}, \quad -1 \text{ a reflection.} \quad (4.2)$$

Key idea

This is exactly the $SO(3)$ structure from the Robotics & Bionics unit: a robot's orientation is an orthogonal matrix with $\det = +1$ (a pure rotation), and its inverse is just its transpose — which is why undoing a rotation is free. The same algebra that orients a surgical tool orients a coordinate frame in PCA.

5 Determinant, Inverse, Rank

The **determinant** of a square matrix measures how it scales area (2-D) or volume (3-D), and its *sign* records orientation. $\det \mathbf{A} = 0$ means the map squashes space into a lower dimension (a line or a point), so it destroys information and **cannot be inverted** — the matrix is *singular*. When $\det \mathbf{A} \neq 0$ the **inverse** $\mathbf{A}^{-1}$ undoes the map, $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$.

The **rank** is the dimension of the output space — the number of independent directions that survive the map. A full-rank $n \times n$ matrix is invertible; a rank-deficient one has collapsed at least one direction (like the projection above). In data terms, rank deficiency means columns (features) are linearly dependent — redundant information — which is precisely what dimensionality reduction exploits.

Exercises

1. Compute $\det \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$ and state the area-scaling factor of the map.
2. Verify that the rotation matrix has determinant 1 and the reflection matrix determinant -1 .
3. Why does the projection onto the x -axis have rank 1 and determinant 0? What does that say about inverting it?

6 Change of Basis

A vector's *coordinates* depend on the basis you measure them in. If the columns of $\mathbf{P}$ are a new set of basis vectors (written in the standard basis), then a point with new-basis coordinates $\mathbf{x}'$ has standard coordinates

$$\mathbf{x} = \mathbf{P}\mathbf{x}', \quad \text{and conversely} \quad \mathbf{x}' = \mathbf{P}^{-1}\mathbf{x}. \quad (6.1)$$

The *same* point, two coordinate systems. A linear map $\mathbf{A}$ expressed in the new basis becomes

$$\mathbf{A}' = \mathbf{P}^{-1}\mathbf{A}\mathbf{P}, \quad (6.2)$$

a **similarity transform**: same map, different coordinates. The art of much of linear algebra — and of PCA — is to *choose a basis in which the map (or the data) is simplest*. For a rotation/orthonormal $\mathbf{P}$, $\mathbf{P}^{-1} = \mathbf{P}^T$, so the change of basis is itself a rigid motion.

Key idea

PCA is exactly a change of basis: it rotates the feature axes to a new orthonormal basis (the principal components) in which the data's covariance is *diagonal* — the features become uncorrelated and ordered by variance. The machinery that finds that basis is the eigendecomposition, next.

7 Eigenvalues and Eigenvectors

Most directions are rotated by a map. An **eigenvector** is a special direction the map only *stretches*, not turns:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}, \quad (7.1)$$

where the scalar λ is its **eigenvalue** (the stretch factor). Eigenvectors are the map's invariant axes; eigenvalues say how much it scales along them. If an $n \times n$ matrix has n independent eigenvectors, stacking them as the columns of $\mathbf{P}$ gives the **diagonalisation**

$$\mathbf{A} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^{-1}, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n), \quad (7.2)$$

which is just (6.2) read backwards: *in the eigenvector basis the map is pure scaling*. This makes powers trivial ($\mathbf{A}^k = \mathbf{P}\mathbf{\Lambda}^k\mathbf{P}^{-1}$) and is the engine behind solving linear dynamical systems — including the state-space models of the Control unit.

The spectral theorem (symmetric matrices) If $\mathbf{A}$ is **symmetric** ($\mathbf{A} = \mathbf{A}^\top$) then its eigenvalues are real and its eigenvectors can be chosen **orthonormal**: $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ with $\mathbf{Q}$ orthogonal. Covariance matrices, Gram matrices and graph Laplacians are all symmetric — so this case is the one ML uses most, and it is what makes PCA's new axes orthogonal.

8 The Singular Value Decomposition

Eigendecomposition needs a square matrix. The **singular value decomposition (SVD)** generalises it to *any* $m \times n$ matrix and is, arguably, the most important factorisation in data science:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top, \quad (8.1)$$

with $\mathbf{U}$ ($m \times m$) and $\mathbf{V}$ ($n \times n$) orthogonal and $\mathbf{\Sigma}$ diagonal with non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. Geometrically every linear map is therefore **rotate** ($\mathbf{V}^\top$) $\rightarrow$ **scale along axes** ($\mathbf{\Sigma}$) $\rightarrow$ **rotate** ($\mathbf{U}$): a rigid turn, an axis-aligned stretch, another rigid turn. The singular values are the lengths of the axes of the ellipse that the unit circle maps to.

Key idea

Low-rank approximation. Keeping only the largest k singular values, $\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, is the *best* rank- k approximation of $\mathbf{A}$ (Eckart–Young). This single fact powers image compression, noise removal, recommender systems and PCA. In the lab you will compress a microscopy-style image by discarding small singular values and watch the picture survive on a fraction of the data — a direct bridge to the image module.

9 Two Payoffs: Least Squares and PCA

9.1 Linear regression is a projection

Fitting $\mathbf{X}\mathbf{w} \approx \mathbf{y}$ when there are more equations than unknowns has no exact solution, so we find the $\mathbf{w}$ making $\mathbf{X}\mathbf{w}$ as close as possible to $\mathbf{y}$ — i.e. the **projection** of $\mathbf{y}$ onto the column space of $\mathbf{X}$. The residual must be orthogonal to every column, $\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, which rearranges to the **normal equation**

$$\boxed{\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y} \quad \Rightarrow \quad \mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.} \quad (9.1)$$

This is the closed-form linear regression of Géron's Chapter 4 — and it is nothing but Section 1's projection, done in many dimensions. (In practice one solves it via SVD, or via the Gram–Schmidt/QR factorisation of Section 1.6, for numerical stability rather than inverting $\mathbf{X}^\top \mathbf{X}$.)

9.2 PCA is an eigen/SVD problem

Principal-component analysis finds the orthonormal directions of greatest variance in centred data $\mathbf{X}$. Those directions are the eigenvectors of the covariance matrix $\frac{1}{m}\mathbf{X}^T\mathbf{X}$ (symmetric — spectral theorem!), equivalently the right singular vectors $\mathbf{V}$ from the SVD of $\mathbf{X}$. The eigenvalues (squared singular values) give the variance captured by each component, so projecting onto the top few $\mathbf{v}_i$ compresses many correlated features into a few uncorrelated ones with minimal information loss — Géron’s Chapter 7. PCA is thus a change of basis (Section 6) chosen by the SVD (Section 8).

Exercises

1. Find the eigenvalues and eigenvectors of $\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$ and of $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. Which directions are invariant?
2. For the symmetric matrix above, verify the eigenvectors are orthogonal (spectral theorem).
3. Explain in one sentence why the normal equation is a projection statement.
4. State what the singular values of a data matrix tell you about its principal components.

10 PageRank: An Eigenvector-Centrality Case Study

Eigenvectors do not just diagonalise a matrix — they can *rank* things. Google’s original PageRank algorithm poses “how important is this web page” as an eigenvalue problem, and the identical algebra ranks genes in a regulatory network or hub regions in a brain-connectivity graph.

10.1 Importance as a fixed point

Model a set of pages as a directed graph, page j pointing at page i whenever it links to it. Define importance recursively: *a page is important if important pages link to it*, each linker splitting its importance evenly across all of its outgoing links. Writing $L_{ij} = 1/(\text{number of pages } j \text{ links to})$ when j links to i and 0 otherwise, the vector of importances $\mathbf{r}$ (nonnegative, summing to 1 — a probability distribution over pages) must satisfy

$$\mathbf{L}\mathbf{r} = \mathbf{r}, \quad (10.1)$$

precisely the eigenvector equation of Section 7 with eigenvalue $\lambda = 1$. Every column of $\mathbf{L}$ sums to 1 (each page redistributes all of its importance), so $\mathbf{L}$ is **column-stochastic** — exactly the transition matrix of a random walk in which a “random surfer” follows a uniformly random outgoing link at each step, and $\mathbf{r}$ is that walk’s **stationary distribution**: the long-run fraction of time the surfer spends on each page.

10.2 Finding $\mathbf{r}$: power iteration

Solving (10.1) directly does not scale to millions of pages. Instead, start from any distribution $\mathbf{r}_0$ and repeatedly apply $\mathbf{L}$,

$$\mathbf{r}_{k+1} = \mathbf{L}\mathbf{r}_k. \quad (10.2)$$

Because $\mathbf{L}$ is column-stochastic its dominant eigenvalue is exactly 1, so this **power iteration** converges (generically) to the $\mathbf{r}$ we want — without ever computing eigenvalues explicitly. It is the same trick used to approximate a dominant eigenvector of any matrix.

10.3 Dead ends, traps and damping

Two structural problems break the simple story. A page with no outgoing links (a **dead end**) has a column of zeros, not a column summing to 1, so it drains probability out of the system every step. A small closed loop of pages (a **trap**) can instead absorb all of the probability and starve everything else. Both are repaired the same way: assume that at every step the surfer follows a link with probability d and otherwise jumps to a uniformly random page,

$$\mathbf{M} = d\mathbf{L} + \frac{1-d}{n}\mathbf{J}, \quad \mathbf{J} = \text{the all-ones matrix}, \quad (10.3)$$

with Google’s own historical choice $d \approx 0.85$. By construction $\mathbf{M}$ is column-stochastic whatever $\mathbf{L}$ looks like, so it always has a well-defined stationary distribution — however the underlying link graph is shaped.

Key idea

This is one instance of **eigenvector centrality**: rank the nodes of *any* weighted graph by “important nodes are those pointed to by important nodes,” i.e. by the dominant eigenvector of its (damped) connectivity matrix. The identical algebra ranks web pages, genes in a regulatory network, or — in the lab — hub regions of a synthetic brain connectivity network, identifying the regions most central to information flow.

Exercises

1. For a 3-page network where every page links to both others (and not to itself), write $\mathbf{L}$ and confirm the uniform distribution $\mathbf{r} = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})^T$ solves (10.1).
2. For a 2-page network where page A links only to B and B links only to A, write $\mathbf{L}$ and confirm $\mathbf{r} = (\frac{1}{2}, \frac{1}{2})^T$ solves (10.1).
3. Explain why a dead-end page breaks the column-stochastic property of $\mathbf{L}$, and how the damping term in (10.3) repairs it.
4. With $d = 0.85$ and $n = 100$ pages, what fraction of a page’s importance in $\mathbf{M}$ comes from the random-jump term rather than from its incoming links?

11 The Lab, and Where This Leads in Géron

Key idea

Companion Colab notebooks. After the theory, the session moves to Colab. Module 2 ships *three* self-contained notebooks — together a complete, biomedical reference, with no need to consult any other material:

- **m2_vectors_transforms** — vectors, norms, dot product and projection; Gram–Schmidt orthonormalization (Section 1.6), including a vectorised speed-up; matrices as maps; the transformation zoo with live 2-D visualisations; orthonormal matrices and rigid motions; determinant and change of basis; and a bonus section reproducing Holbein’s anamorphic-skull illusion as a worked change of basis;
- **m2_eigen_svd_pca** — eigenvectors and diagonalisation; the SVD geometry; image compression by low-rank SVD; least squares via the normal equation; and PCA on the breast-cancer data, compared to Scikit-Learn;
- **m2_pagerank_networks** — PageRank as an eigenvalue problem (Section 10); power iteration; dead ends, traps and the damping parameter; and a bioengineering capstone

ranking hub regions of a synthetic brain-connectivity network.

The first two are each a superset of Géron’s `math_linear_algebra` notebook, re-ordered around geometry and worked on biomedical data; all three end their sections with exercises and worked solutions.

Where this unlocks the book. Projection gives the normal equation of **Chapter 4** (training linear models); Gram–Schmidt/QR gives the numerically stable way to actually solve it; the eigen/SVD machinery gives **Chapter 7** (dimensionality reduction / PCA); matrices-as-maps is exactly a neural-network layer in **Chapters 9–10**; and PageRank is the same eigenvector idea turned into a ranking algorithm — the ancestor of every “importance score” built on a graph. *Cross-links:* the rotation and orthonormal-matrix material is the $SO(3)$ of the Robotics & Bionics unit, and the eigen-machinery reappears in the state-space analysis of the Control unit.

The one-paragraph summary

Read a matrix as a verb: its columns say where the basis goes. The dot product measures length, angle and overlap, and projection splits a vector into “along” and “perpendicular” parts — which is least squares; Gram–Schmidt repeats that split to build an orthonormal basis, and hence a QR factorisation, out of any one. Orthogonal matrices are rigid motions ($\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$, $\det = \pm 1$); the determinant is the area/volume scale and zero means non-invertible. Eigenvectors are the directions a map only stretches; for symmetric matrices they are orthonormal, and the same fixed-point idea — iterated on a column-stochastic matrix instead of diagonalised — is PageRank. The SVD writes any map as rotate–scale–rotate and gives the best low-rank approximation — the basis of compression and of PCA. Choose the right basis and the problem becomes easy.